

```
"""
```

```
calibration_ds_method.py
```

```
=====
```

```
Calibration de la mesure d_s sur systèmes à dimension spectrale connue  
théoriquement (réseaux réguliers), puis application au Blob.
```

```
Méthodes testées en parallèle :
```

```
  M1 - Comptage d'eigenvalues :  $N(\lambda) \propto \lambda^{(d_s/2)}$  sur le bas du spectre  $L = D-A$ 
```

```
  M2 - Heat kernel trace      :  $\text{Tr}(e^{-tL})/N \propto t^{-d_s/2}$  à t intermédiaire
```

```
Références théoriques :
```

```
  - 1D chain                  : d_s = 1
```

```
  - 2D square lattice       : d_s = 2
```

```
  - 3D cubic lattice        : d_s = 3
```

```
"""
```

```
import numpy as np
```

```
import math, time
```

```
from collections import deque
```

```
from scipy.spatial import KDTree
```

```
from scipy.linalg import eigvalsh
```

```
from scipy.stats import linregress
```

```
# ===== Construction réseaux =====
```

```
def build_1d_chain(N):
```

```
    adj = np.zeros((N, N), dtype=np.int8)
```

```
    for i in range(N - 1):
```

```
        adj[i, i+1] = adj[i+1, i] = 1
```

```
    return adj
```

```
def build_2d_lattice(L):
```

```
    N = L * L
```

```
    adj = np.zeros((N, N), dtype=np.int8)
```

```
    for i in range(L):
```

```
        for j in range(L):
```

```
            v = i * L + j
```

```
            if i + 1 < L:
```

```
                u = (i+1) * L + j; adj[v, u] = adj[u, v] = 1
```

```
            if j + 1 < L:
```

```
                u = i * L + (j+1); adj[v, u] = adj[u, v] = 1
```

```
    return adj
```

```
def build_3d_lattice(L):
```

```
    N = L * L * L
```

```

adj = np.zeros((N, N), dtype=np.int8)
def idx(i, j, k): return i*L*L + j*L + k
for i in range(L):
    for j in range(L):
        for k in range(L):
            v = idx(i, j, k)
            if i+1 < L: u = idx(i+1, j, k); adj[v, u] = adj[u, v] = 1
            if j+1 < L: u = idx(i, j+1, k); adj[v, u] = adj[u, v] = 1
            if k+1 < L: u = idx(i, j, k+1); adj[v, u] = adj[u, v] = 1
return adj

# ===== Construction Blob (port O2v4 minimal) =====
CFG = {"target_degree": 8.0, "proposal_radius_mult": 1.5,
       "burn_in_factor": 100, "prod_factor": 200,
       "tau": 0.50, "tau_scale_N": True, "beta": 0.30}

def _rips_radius(N, cfg=CFG):
    return (cfg["target_degree"] / ((N-1) * (4/3) * math.pi)) ** (1/3)

def _E0(N, cfg=CFG): return int(N * cfg["target_degree"] / 2)

def _lcc_size(adj_sets, N):
    visited = [False]*N; best = 0
    for s in range(N):
        if not visited[s]:
            sz = 0; q = deque([s]); visited[s] = True
            while q:
                v = q.popleft(); sz += 1
                for w in adj_sets[v]:
                    if not visited[w]: visited[w] = True; q.append(w)
            best = max(best, sz)
    return best

def build_blob_equilibrium(N, seed, cfg=CFG):
    r = _rips_radius(N); r_p = cfg["proposal_radius_mult"] * r
    E0 = _E0(N)
    rng = np.random.default_rng(seed)
    pts = rng.uniform(0.0, 1.0, (N, 3))
    tree = KDTree(pts)
    adj = [set() for _ in range(N)]
    for i, j in tree.query_pairs(r):
        adj[i].add(j); adj[j].add(i)
    pool = list(tree.query_pairs(r_p))
    rng2 = np.random.default_rng(seed + 50_000)
    K = sum(len(adj[i]) for i in range(N)) // 2
    att = 0; max_att = 10 * abs(E0 - K) + 5000
    while K != E0 and att < max_att:

```

```

    att += 1
    if K < E0:
        e = pool[rng2.integers(len(pool))]; u, v = e
        if v not in adj[u]: adj[u].add(v); adj[v].add(u); K += 1
    else:
        edges = [(i,j) for i in range(N) for j in adj[i] if j>i]
        if not edges: break
        e = edges[rng2.integers(len(edges))]; u, v = e
        adj[u].discard(v); adj[v].discard(u)
        if _lcc_size(adj, N) == N: K -= 1
        else: adj[u].add(v); adj[v].add(u)

# MCMC
tau = cfg["tau"] / N
rng3 = np.random.default_rng(seed + 200_000)
edge_list = [(i,j) for i in range(N) for j in adj[i] if j>i]
def ntri():
    n = 0
    for i in range(N):
        for j in adj[i]:
            if j > i:
                n += len([k for k in adj[i] & adj[j] if k > j])
    return n
def S(n_tri):
    degs = np.array([len(adj[i]) for i in range(N)], float)
    return math.log(1+n_tri) - cfg["beta"] * degs.std()/(degs.mean()+1e-8)
n_tri = ntri(); S_curr = S(n_tri)
n_steps = (cfg["burn_in_factor"] + cfg["prod_factor"]) * N
u_arr = rng3.random(size=n_steps)
pool_arr = np.array(pool, dtype=np.int32); n_pool = len(pool_arr)
for step in range(n_steps):
    if not edge_list: break
    idx_out = int(rng3.integers(len(edge_list)))
    u, v = edge_list[idx_out]
    e_in = None
    for _ in range(30):
        idx_in = int(rng3.integers(n_pool))
        a, b = int(pool_arr[idx_in,0]), int(pool_arr[idx_in,1])
        if b not in adj[a]: e_in = (a,b); break
    if e_in is None: continue
    a, b = e_in
    tri_lost = len(adj[u] & adj[v])
    adj[u].discard(v); adj[v].discard(u)
    adj[a].add(b); adj[b].add(a)
    tri_gained = len(adj[a] & adj[b])
    n_tri_new = n_tri - tri_lost + tri_gained
    S_new = S(n_tri_new); dS = S_new - S_curr
    if dS >= 0 or u_arr[step] < math.exp(dS / tau):

```

```

        n_tri = n_tri_new; S_curr = S_new
        edge_list[idx_out] = (a, b)
    else:
        adj[u].add(v); adj[v].add(u); adj[a].discard(b); adj[b].discard(a)
# Adjacency matrix
A = np.zeros((N, N), dtype=np.int8)
for i in range(N):
    for j in adj[i]: A[i, j] = 1
return A, n_tri

# ===== Méthodes de mesure d_s =====
def measure_ds_eigvals(adj, threshold_frac=0.15):
    N = adj.shape[0]
    deg = adj.sum(axis=1).astype(np.float64)
    L = np.diag(deg) - adj.astype(np.float64)
    evals = eigvalsh(L)
    evals = np.sort(evals)
    evals = evals[evals > 1e-9]
    if len(evals) < 20: return None, None, 0
    lam_max = evals.max()
    threshold = threshold_frac * lam_max
    mask = evals <= threshold
    n_low = int(mask.sum())
    if n_low < 10: n_low = max(20, len(evals)//4)
    lam_arr = evals[:n_low]
    N_arr = np.arange(1, n_low + 1)
    res = linregress(np.log(lam_arr), np.log(N_arr))
    return float(2 * res.slope), float(res.rvalue**2), n_low

def measure_ds_heat_kernel(adj):
    """d_s via  $\text{Tr}(e^{-tL})/N \sim t^{-d_s/2}$  dans la région intermédiaire."""
    N = adj.shape[0]
    deg = adj.sum(axis=1).astype(np.float64)
    L = np.diag(deg) - adj.astype(np.float64)
    evals = eigvalsh(L)
    evals_pos = evals[evals > 1e-9]
    if len(evals_pos) < 20: return None, None
    lam_min = evals_pos.min(); lam_max = evals_pos.max()
    # t intermédiaire : éviter saturation aux deux bouts
    t_arr = np.logspace(np.log10(1.0/lam_max * 3), np.log10(1.0/lam_min / 3), 40)
    K_arr = np.array([np.mean(np.exp(-evals * t)) for t in t_arr])
    # Garder la région où K n'est ni saturée à 1/N (haut t) ni à 1 (bas t)
    valid = (K_arr > 2.0/N) & (K_arr < 0.7)
    if valid.sum() < 5: return None, None
    t_v = t_arr[valid]; K_v = K_arr[valid]
    res = linregress(np.log(t_v), np.log(K_v))
    return float(-2 * res.slope), float(res.rvalue**2)

```

```

# ===== Main =====
if __name__ == "__main__":
    print("=" * 80)
    print("CALIBRATION DE LA MÉTHODE d_s - réseaux à dimension connue + Blob")
    print("=" * 80)
    print(f"{'Système':<25}{'N':>6}{'d_s théo':>10}{'d_s eigvals':>14}{'(R²)':>7}"
          f"{'d_s heat':>11}{'(R²)':>7}")
    print("-" * 80)

    systems = [
        ("1D chain",          build_1d_chain(1000), 1.0),
        ("2D lattice (30×30)", build_2d_lattice(30), 2.0),
        ("3D cubic (10×10×10)", build_3d_lattice(10), 3.0),
    ]
    for label, adj, d_theo in systems:
        t0 = time.time()
        d_eig, r2_eig, n_low = measure_ds_eigvals(adj)
        d_hk, r2_hk = measure_ds_heat_kernel(adj)
        N = adj.shape[0]
        ts = time.time() - t0
        eig_str = f"{d_eig:.3f}" if d_eig is not None else "FAIL"
        hk_str = f"{d_hk:.3f}" if d_hk is not None else "FAIL"
        print(f"{label:<25}{N:>6}{d_theo:>10.2f}{eig_str:>14}({r2_eig:.3f}){hk_str:>11}"

# ----- Blob -----
print()
print("Blob (équilibre O2v4 Phase A) :")
print(f"{'Seed':<8}{'N':>6}{'n_tri':>8}{'d_s eigvals':>14}{'(R²)':>7}{'d_s he
print("-" * 70)
seeds = [0, 1, 2]
blob_results = {'eig': [], 'hk': []}
for seed in seeds:
    t0 = time.time()
    adj, n_tri = build_blob_equilibrium(500, seed)
    d_eig, r2_eig, _ = measure_ds_eigvals(adj)
    d_hk, r2_hk = measure_ds_heat_kernel(adj)
    ts = time.time() - t0
    blob_results['eig'].append(d_eig)
    blob_results['hk'].append(d_hk)
    eig_str = f"{d_eig:.3f}" if d_eig is not None else "FAIL"
    hk_str = f"{d_hk:.3f}" if d_hk is not None else "FAIL"
    print(f"{seed:<8}{500:>6}{n_tri:>8}{eig_str:>14}({r2_eig:.3f}){hk_str:>11}"

print()
if blob_results['eig']:
    m_eig = np.mean(blob_results['eig']); s_eig = np.std(blob_results['eig'])

```

```
m_hk = np.mean(blob_results['hk']); s_hk = np.std(blob_results['hk'])
print(f"Blob moyenne eigvals = {m_eig:.3f} ± {s_eig:.3f}")
print(f"Blob moyenne heat      = {m_hk:.3f} ± {s_hk:.3f}")

print("\n" + "=" * 80)
print("VERDICT")
print("=" * 80)
```